

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск набора подстрок в строке. Вариант 2.

Студент гр. 4345

Лунченко Н.С.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы:

Изучить Алгоритм Ахо-Корасика. Подсчитать количество вершин в автомате; вывести список найденных образцов, имеющих пересечения с другими найденными образцами в строке поиска.

Постановка задачи

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст ($T, 1 \leq |T| \leq 100000$).

Вторая - число ($1 \leq n \leq 3000$), каждая следующая из n строк содержит шаблон из набора

$$P = \{p_1, \dots, p_n\} \quad 1 \leq |p_i| \leq 75$$

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Выход:

Все вхождения образцов из P в T .

Каждое вхождение образца в текст представить в виде двух чисел – i p .
Где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p
(нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

Input data

NTAG

3

TAGT

TAG

T

Output data

2 2

2 3

Выполнение работы

Базовое представление данных

Автомат Ахо-Корасик представлен в виде набора массивов, где каждый индекс соответствует вершине бора. Для хранения переходов используется список словарей, что обеспечивает эффективный доступ к исходящим дугам по символу за $O(1)$ в среднем.

Реализованные структуры данных:

Класс AhoCorasick содержит следующие компоненты для каждой вершины:

next - список словарей переходов по символам (бор)

fail - суффиксные ссылки (переход при несовпадении)

term - конечные (терминальные) ссылки для быстрого перехода к ближайшей терминальной вершине

out - списки образцов, заканчивающихся в данной вершине

vertices - счетчик количества вершин в автомате

Основные алгоритмы

Добавление образца в бор (add_pattern)

Выполняется посимвольный обход образца, начиная с корневой вершины (0). Для каждого символа проверяется наличие перехода: если переход существует, выполняется переход по нему; если отсутствует - создается новая вершина с инициализацией всех массивов. В конечной вершине в out добавляется идентификатор образца.

Временная сложность: $O(m)$, где m - длина образца.

Построение автомата (build_automaton)

Выполняется обход бора в ширину (BFS) с использованием очереди. Для каждой вершины обрабатываются все исходящие дуги:

Инициализация: для всех детей корня суффиксная ссылка устанавливается на корень

Для каждой вершины-потомка вычисляется суффиксная ссылка путем подъема по суффиксным ссылкам предка до нахождения совпадающего перехода или достижения корня

Конечная ссылка: если суффиксная ссылка указывает на терминальную вершину, конечная ссылка устанавливается на нее; иначе наследуется конечная ссылка от суффиксной

Временная сложность: $O(K * |\Sigma|)$, где K - количество вершин, $|\Sigma|$ - размер алфавита.

Функция перехода (`get_next`)

Реализует ленивое вычисление переходов с кэшированием. Если переход по символу отсутствует в текущей вершине, то:

для корня устанавливается переход на корень

для остальных вершин рекурсивно вычисляется переход из суффиксной ссылки

Результат кэшируется в `next[v][ch]`.

Временная сложность: $O(1)$ амортизированная.

Поиск образцов (`search`)

Выполняется посимвольный обход текста. Для каждого символа:

Выполняется переход в следующую вершину автомата

Проверяется текущая вершина на наличие образцов

По цепочке конечных ссылок собираются все найденные образцы

Для каждого образца вычисляется позиция начала в тексте

Временная сложность: $O(n + k)$, где n - длина текста, k - количество найденных вхождений.

Анализ пересечений (`find_overlapping`)

Для каждой пары найденных вхождений проверяется пересечение их интервалов `[start, end]`. Интервалы пересекаются, если не выполняется условие $(end1 < start2 \text{ или } end2 < start1)$. Пересекающиеся образцы собираются в множество и выводятся отсортированными.

Временная сложность: $O(k^2)$, где k - количество найденных вхождений.

Пошаговое описание работы

Инициализация:

Чтение текста и количества образцов

Создание экземпляра автомата с корневой вершиной

Инициализация массивов: $next = [\{\}], fail = [0], term = [-1], out = [[]]$

Построение бора:

Для каждого образца вызывается `add_pattern`

Образец "TAGT": создаются вершины $1 \Rightarrow 2 \Rightarrow 3 \Rightarrow 4$, $out[4] = [1]$

Образец "TAG": используется существующий путь, $out[3] = [2]$

Образец "T": используется вершина 1, $out[1] = [3]$

Построение автомата:

BFS обход: вершины 1, 2, 3, 4

$fail[1] = 0$ (ребенок корня)

$fail[2] = 0$ (нет перехода 'A' из корня)

$fail[3] = 0$ (нет перехода 'G' из корня)

$fail[4] = 1$ (переход 'T' из корня в вершину 1)

$term[4] = 1$ ($fail[4]$ содержит образец 3)

Поиск в тексте "NTAG":

Позиция 1, 'N': переход в 0 (нет перехода)

Позиция 2, 'T': переход в 1, найден образец 3, $term[1] = -1$

Позиция 3, 'A': переход в 2, $term[2] = -1$

Позиция 4, 'G': переход в 3, найден образец 2, $term[3] = -1$

Анализ пересечений

Образец 2 (поз. 2-4) и образец 3 (поз. 2-2) имеют пересечение

Эвристики и оптимизации

Конечные ссылки: обеспечивают переход сразу к ближайшей терминальной вершине без обхода всей цепочки суффиксных ссылок, что гарантирует линейную сложность поиска $O(n + k)$.

Кэширование переходов: функция `get_next` запоминает вычисленные переходы, что позволяет избежать повторных вычислений и обеспечивает амортизированное $O(1)$ на переход.

Хранение переходов в словарях: обеспечивает компактное представление для разреженного бора и быстрый доступ по символу.

Оценка сложности алгоритма

Сложность по времени:

Построение бора: $O(\sum |P_i|)$, где $|P_i|$ - длины образцов

Построение автомата: $O(K * |\Sigma|)$, где K - количество вершин

Поиск: $O(n + k)$, где n - длина текста, k - количество вхождений

Анализ пересечений: $O(k^2)$

Сложность по памяти:

$O(K * |\Sigma|)$ для хранения переходов

$O(K)$ для хранения суффиксных и конечных ссылок

$O(\sum |P_i|)$ для хранения образцов

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1.

Таблица 1 – Результаты тестирования

№	Входные данные	Выходные данные	Комментарий
1	NTAG 3 TAGT TAG T	Количество вершин: 5 Вхождения: (2, 2), (2, 3) Пересечения: образец 2 (TAG) и образец 3 (T)	Базовый тест из условия
2	ACTGACTGACTG 4 ACTG, CTG, TG, G	Количество вершин: 11 Вхождения: (1,1), (2,2), (3,3), (4,4), (5,1), (6,2),	В тесте 2 образцы ACTG, CTG, TG и G имеют разные первые символы (A, C, T, G), поэтому в боре создаются отдельные

		(7,3), (8,4), (9,1), (10,2), (11,3), (12,4) Пересечения: все образцы в каждом блоке (1-4, 5-8, 9-12)	ветви от корня - всего 11 вершин. Благодаря цепочке терминальных ссылок $term[4] \Rightarrow term[7] \Rightarrow term[9] \Rightarrow term[10]$ все четыре образца эффективно находятся за один проход по тексту в каждом из трёх повторяющихся блоков.
3	AAAAA 3 A, AA, AAA	Количество вершин: 4 Вхождения: (1,1), (1,2), (1,3), (2,1), (2,2), (2,3), (3,1), (3,2), (3,3), (4,1), (4,2), (5,1) Пересечения: все образцы на всех позициях	Образцы A, AA, AAA имеют общий префикс, поэтому бор строится как одна цепочка $0 \Rightarrow 1 \Rightarrow 2 \Rightarrow 3$, что даёт всего 4 вершины. Терминальные ссылки образуют цепочку $term[3]=2$, $term[2]=1$, позволяя на каждой позиции после прочтения символа находить до трёх образцов за один проход.
4	TCGATCGAATCG 5 TCG, CGA, ATC, GAA, CGAA	Количество вершин: 14 Вхождения: (1,1), (2,2), (4,3), (5,1), (6,2), (6,5), (7,4), (9,3), (10,1) Пересечения: образцы на позициях 1-4, 4-9, 9-12	Процесс поиска демонстрирует эффективность терминальных ссылок: На позиции 4 после нахождения образца 1 (ACTG) алгоритм переходит по цепочке $term[4] \Rightarrow term[7] \Rightarrow term[9] \Rightarrow term[10]$, находя образцы 2, 3, 4 за $O(1)$ каждый, вместо обхода всех суффиксных ссылок.

Выполнение работы

Постановка задачи

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с джокером.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблон образцу P необходимо найти все вхождения P в текст T .

Например, образец $ab??c?$ с джокером $?$ встречается дважды в тексте $xabvccbababcsax$.

Символ джокер не входит в алфавит, символы которого используются в T . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида $???$ недопустимы.

Все строки содержат символы из алфавита $\{A,C,G,T,N\}$

Вход:

Текст ($T, 1 \leq |T| \leq 100000$)

Шаблон ($P, 1 \leq |P| \leq 40$)

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

Test data

1

Input data

ACTANCA

A\$\$\$A\$

\$

Output data

1.

Выполнение работы (Задание 2: Поиск с джокером)

Базовое представление данных

Для решения задачи поиска образца с джокерами используется алгоритм Ахо-Корасик в комбинации с методом скользящего счетчика. Образец разбивается на подобразцы по символам джокера, после чего выполняется множественный поиск всех подобразцов в тексте. Совпадение полного образца определяется по одновременному нахождению всех подобразцов на правильных относительных позициях.

Реализованные структуры данных:

Класс AhoCorasick для поиска подобразцов:

nexth - список словарей переходов по символам (бор)

fail - суффиксные ссылки для переходов при несовпадении

term - конечные ссылки для быстрого перехода к терминальным вершинам

out - списки идентификаторов подобразцов, заканчивающихся в вершине

vertices - счетчик количества вершин в автомате

Вспомогательные структуры для обработки джокеров:

subpatterns - список подобразцов без джокеров

positions - список позиций подобразцов в исходном образце

counter - массив счетчиков совпадений для каждой позиции в тексте

Основные алгоритмы

Разбиение образца на подобразцы

Выполняется однопроходный обход образца. При обнаружении символа джокера текущая накапливаемая подстрока сохраняется как подобразец с запоминанием её начальной позиции в образце. Подобразцы, следующие друг за другом без джокеров, объединяются в один подобразец.

Временная сложность: $O(|P|)$, где $|P|$ - длина образца.

Поиск подобразцов (search)

Используется алгоритм Ахо-Корасик для поиска всех вхождений подобразцов в тексте. Для каждого найденного подобразца сохраняется его позиция в тексте и идентификатор.

Временная сложность: $O(|T| + K)$, где $|T|$ - длина текста, K - количество найденных вхождений подобразцов.

Проверка совпадений с джокерами

Для каждого найденного подобразца вычисляется предполагаемая позиция начала полного образца: $start = pos - positions[pat_id]$. Если позиция допустима (образец помещается в текст), увеличивается счетчик совпадений для этой позиции. Когда счетчик достигает количества подобразцов, фиксируется полное совпадение образца с джокерами.

Временная сложность: $O(K)$, где K - количество найденных вхождений подобразцов.

Пошаговое описание работы

Шаг 1: Разбиение образца на подобразцы

Образец A\$\$A\$ разбивается на подобразцы:

A на позиции 0

A на позиции 3

Длина полного образца: 5

Шаг 2: Построение бора для подобразцов

Подобразец A (ID=1): вершина 0 => вершина 1 по символу 'A'

Подобразец A (ID=2): используется существующий переход в вершину

1

Оба подобразца заканчиваются в вершине 1: $out[1] = [1, 2]$

Шаг 3: Построение автомата

BFS обход: вершина 1 (нет исходящих дуг, терминальная)

$fail[1] = 0$ (ребенок корня)

$term[1] = -1$ ($fail$ указывает на корень, нетерминальный)

Шаг 4: Поиск подобразцов в тексте "ASTANCA"

Позиция 1, 'A': переход в 1, найдены подобразцы 1 и 2 на позиции 0

Позиция 2, 'C': переход в 0

Позиция 3, 'T': переход в 0

Позиция 4, 'A': переход в 1, найдены подобразцы 1 и 2 на позиции 3

Позиция 5, 'N': переход в 0

Позиция 6, 'C': переход в 0

Позиция 7, 'A': переход в 1, найдены подобразцы 1 и 2 на позиции

Шаг 5: Проверка полных совпадений

Подобразец 1 на позиции 0: $\text{start} = 0 - 0 = 0$, $\text{counter}[0] = 1/2$

Подобразец 2 на позиции 0: $\text{start} = 0 - 3 = -3$ (выход за границы)

Подобразец 1 на позиции 3: $\text{start} = 3 - 0 = 3$ ($3 + 5 > 7$, выход за границы)

Подобразец 2 на позиции 3: $\text{start} = 3 - 3 = 0$, $\text{counter}[0] = 2/2$

СОВПАДЕНИЕ на позиции 1

Подобразец 1 на позиции 6: $\text{start} = 6 - 0 = 6$ ($6 + 5 > 7$, выход за границы)

Подобразец 2 на позиции 6: $\text{start} = 6 - 3 = 3$ ($3 + 5 > 7$, выход за границы)

Шаг 6: Визуальная проверка совпадения

text

Текст: A C T A N C A

Образец: A \$ \$ A \$

Позиции: 1 2 3 4 5

Образец A\$\$\$ совпадает с АСТАН на позиции 1:

A = A (символ)

\$ = C (джокер совпадает с любым символом)

\$ = T (джокер совпадает с любым символом)

A = A (символ)

\$ = N (джокер совпадает с любым символом)

Эвристики и оптимизации

Объединение смежных подобразцов: символы без джокеров, идущие подряд, объединяются в один подобразец, что уменьшает количество подобразцов для поиска и повышает эффективность.

Кэширование переходов: функция `get_next` запоминает вычисленные переходы, обеспечивая амортизированную сложность $O(1)$ на символ текста.

Конечные ссылки: используются для быстрого перехода между терминальными вершинами без обхода всех суффиксных ссылок, что сохраняет линейную сложность поиска.

Массив счетчиков: используется для эффективной проверки одновременного нахождения всех подобразцов на правильных позициях за $O(1)$ на каждое вхождение подобразца.

Оценка сложности алгоритма

Сложность по времени:

Разбиение образца: $O(|P|)$

Построение бора: $O(\sum |S_i|)$, где $|S_i|$ - длины подобразцов

Построение автомата: $O(V * |\Sigma|)$, где V - количество вершин

Поиск подобразцов: $O(|T| + K)$, где K - количество вхождений подобразцов

Проверка совпадений: $O(K)$

Общая сложность: $O(|P| + |T| + V * |\Sigma| + K)$

Сложность по памяти:

$O(V * |\Sigma|)$ для хранения автомата

$O(|T|)$ для массива счетчиков

$O(K)$ для хранения найденных вхождений подобразцов Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 2.

Таблица 2 – Результаты тестирования

№	Входные данные	Выходные данные	Комментарий
1	Текст: АСТАНСА, Образец: А\$\$А,Джокер:.,Джокер:	1	Базовый тест из условия. Образец разбит на подобразцы 'А' (поз. 0) и 'А' (поз. 3). Полное совпадение

			найден на позиции 1: A\$\$\$ = ACTAN.
2	Текст: NTAG, Образец: NAG, Джокер: AG, Джокер:	1	Джокер в середине образца. Подобразцы 'N' (поз. 0) и 'AG' (поз. 2). Совпадение на позиции 1: N\$AG = NTAG.
3	Текст: AAAAA, Образец: AA, Джокер: A, Джокер:	1, 2, 3	Образец с джокером и повторяющимся текстом. Подобразцы 'A' (поз. 0) и 'A' (поз. 2). Найдены совпадения на позициях 1, 2, 3.
4	Текст: TCGATCGAATCG, Образец: TCG\$\$CGA, Джокер: \$	1	Сложный образец с двумя джокерами. Подобразцы 'TCG' (поз. 0) и 'CGA' (поз. 5). Совпадение на позиции 1: TCG\$\$CGA = TCGATCGA.

Вывод

В ходе работы реализован алгоритм Ахо-Корасик для точного множественного поиска и поиска с джокером с полным выводом промежуточных данных. Для задания 1 выполнен вариант 2: реализован подсчёт количества вершин в автомате и определён список образцов, имеющих пересечения в тексте. Тестирование на четырёх наборах данных подтвердило корректность построения бора, суффиксных и терминальных ссылок. Благодаря использованию конечных ссылок поиск выполняется за линейное время $O(n + k)$ без обхода всех суффиксных связей. Результаты работы демонстрируют эффективность алгоритма для задач множественного поиска и поиска с джокерами.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл 5_1.py

```
class AhoCorasick:
    def __init__(self):
        self.next = [{}]
        self.fail = [0]
        self.term = [-1]
        self.out = [[]]
        self.vertices = 1

    def add_pattern(self, pattern, pattern_id):
        print(f"\nДобавление образца '{pattern}' (ID={pattern_id + 1})")

        v = 0
        for i, ch in enumerate(pattern):
            print(f"Текущая вершина: {v}, символ: '{ch}'")
            if ch not in self.next[v]:
                new_vertex = self.vertices
                self.next[v][ch] = new_vertex
                self.next.append({})
                self.fail.append(0)
                self.term.append(-1)
                self.out.append([])
                self.vertices += 1
                print(f"Создана новая вершина {new_vertex}")
            else:
                print(f"Переход по существующему ребру в вершину {self.next[v][ch]}")
                v = self.next[v][ch]

            self.out[v].append(pattern_id)
            print(f"Образец {pattern_id + 1} заканчивается в вершине {v}")
            print(f"Текущий бор: {self.next}")

    def build_automaton(self):
        print(f"\nПОСТРОЕНИЕ АВТОМАТА")
        print(f"Начальный бор: {self.next}")
        print(f"Количество вершин в боре: {self.vertices}")

        queue = []

        print(f"\n1. Инициализация суффиксных ссылок для детей корня:")
        for ch, nxt in self.next[0].items():
            self.fail[nxt] = 0
            queue.append(nxt)
            print(f"Вершина {nxt}: fail = 0 (корень)")
```

```

        head = 0
        print(f"\nBFS    построение    суффиксных    и    конечных
ссылок:")
        while head < len(queue):
            v = queue[head]
            head += 1
            print(f"\n    Обработка вершины {v}:")
            print(f"        Исходящие ребра: {self.next[v]}")

            for ch, nxt in self.next[v].items():
                print(f"        Символ '{ch}' -> вершина {nxt}:")

                f = self.fail[v]
                print(f"        Поиск суффиксной ссылки,
начиная с fail[{v}] = {f}")

                while f > 0 and ch not in self.next[f]:
                    print(f"        Переход по fail[{f}] =
{self.fail[f]}")
                    f = self.fail[f]

                if ch in self.next[f]:
                    self.fail[nxt] = self.next[f][ch]
                    print(f"        Найдено: fail[{nxt}] =
{self.fail[nxt]}")
                else:
                    self.fail[nxt] = 0
                    print(f"        Не найдено: fail[{nxt}] =
0 (корень)")

                    if self.out[self.fail[nxt]]:
                        self.term[nxt] = self.fail[nxt]
                        print(f"        Терминальная ссылка:
term[{nxt}] = {self.term[nxt]} (выход в fail)")
                    else:
                        self.term[nxt] =
self.term[self.fail[nxt]]
                        print(f"        Терминальная ссылка:
term[{nxt}] = {self.term[nxt]} (наследована)")

                    queue.append(nxt)

        print(f"\n3. ПОСТРОЕННЫЙ АВТОМАТ:")
        for i in range(self.vertices):
            print(f"Вершина {i}:")
            print(f"    Переходы: {self.next[i]}")
            print(f"    Суффиксная ссылка: {self.fail[i]}")
            print(f"    Терминальная ссылка: {self.term[i]}")
            print(f"    Выходные образцы: {[p + 1 for p in
self.out[i]]}")

        def get_next(self, v, ch):

```

```

        if ch not in self.next[v]:
            if v == 0:
                self.next[v][ch] = 0
            else:
                self.next[v][ch]
self.get_next(self.fail[v], ch)
        return self.next[v][ch]

def search(self, text, patterns):
    print(f"\nПРОЦЕСС ПОИСКА В ТЕКСТЕ: '{text}'")

    results = []
    v = 0
    print(f"Начальное состояние: вершина {v}")

    for i, ch in enumerate(text):
        print(f"\nПозиция {i + 1}, СИМВОЛ '{ch}':")
        v = self.get_next(v, ch)
        print(f"  Переход в вершину: {v}")
        u = v

        if u > 0:
            print(f"  Проверка терминальных вершин:")

            while u > 0:
                if self.out[u]:
                    for pattern_id in self.out[u]:
                        pattern_len
len(patterns[pattern_id])
                        pos = i - pattern_len + 1
                        end_pos = i
                        print(f"                Найден образец
{pattern_id + 1} на позиции {pos + 1}-{end_pos + 1}")
                        results.append((pos + 1, pattern_id
+ 1, pos, end_pos))
                print(f"                Переход по term[{u}] =
{self.term[u]}")
                u = self.term[u]

    return results

def find_overlapping(results):
    print(f"\nАНАЛИЗ ПЕРЕСЕЧЕНИЙ:")
    overlapping = set()

    for i in range(len(results)):
        pos1, pat_id1, start1, end1 = results[i]
        for j in range(i + 1, len(results)):
            pos2, pat_id2, start2, end2 = results[j]

            if pat_id1 == pat_id2:
                continue

```

```

        if not (end1 < start2 or end2 < start1):
            print(
                f"        Пересечение: образец {pat_id1}
(поз.{pos1}-{end1 + 1}) и образец {pat_id2} (поз.{pos2}-{end2 +
1}))")
            overlapping.add((pos1, pat_id1))
            overlapping.add((pos2, pat_id2))

    return sorted(overlapping)

if __name__ == "__main__":
    text = input().strip()
    n = int(input())
    patterns = []

    ac = AhoCorasick()

    for i in range(n):
        pattern = input().strip()
        patterns.append(pattern)
        ac.add_pattern(pattern, i)

    ac.build_automaton()

    print(f"Количество вершин в автомате: {ac.vertices}")

    results = ac.search(text, patterns)

    results.sort(key=lambda x: (x[0], x[1]))

    print(f"\nВСЕ НАЙДЕННЫЕ ВХОЖДЕНИЯ:")
    for pos, pat_id, _, _ in results:
        print(f"Позиция      {pos},      образец      {pat_id}:
{patterns[pat_id - 1]}")

    overlapping = find_overlapping(results)

    print(f"\nОБРАЗЦЫ, ИМЕЮЩИЕ ПЕРЕСЕЧЕНИЯ:")
    if overlapping:
        for pos, pat_id in overlapping:
            print(f"Позиция      {pos},      образец      {pat_id}:
{patterns[pat_id - 1]}")
    else:
        print("Пересекающиеся образцы не найдены")

```

Файл 5_2.py

```

class AhoCorasick:
    def __init__(self):
        self.next = [{}]

```

```

        self.fail = [0]
        self.term = [-1]
        self.out = [[]]
        self.vertices = 1

    def add_pattern(self, pattern, pattern_id):
        print(f"\nДобавление подобразца '{pattern}'
(ID={pattern_id + 1})")

        v = 0
        for i, ch in enumerate(pattern):
            print(f" Текущая вершина: {v}, символ: '{ch}'")
            if ch not in self.next[v]:
                new_vertex = self.vertices
                self.next[v][ch] = new_vertex
                self.next.append({})
                self.fail.append(0)
                self.term.append(-1)
                self.out.append([])
                self.vertices += 1
                print(f" Создана новая вершина
{new_vertex}")
            else:
                print(f" Переход по существующему ребру в
вершину {self.next[v][ch]}")
                v = self.next[v][ch]

            self.out[v].append(pattern_id)
            print(f" Подобразец {pattern_id + 1} заканчивается
в вершине {v}")

    def build_automaton(self):
        print(f"\nПОСТРОЕНИЕ АВТОМАТА")
        print(f"Начальный бор: {self.next}")
        print(f"Количество вершин в боре: {self.vertices}")

        queue = []

        print(f"\nИнициализация суффиксных ссылок для детей
корня:")

        for ch, nxt in self.next[0].items():
            self.fail[nxt] = 0
            queue.append(nxt)
            print(f" Вершина {nxt}: fail = 0 (корень)")

        head = 0
        print(f"\nBFS построение суффиксных и конечных
ссылок:")

        while head < len(queue):
            v = queue[head]
            head += 1
            print(f"\n Обработка вершины {v}:")
            print(f" Исходящие ребра: {self.next[v]}")

```

```

        for ch, nxt in self.next[v].items():
            print(f"    Символ '{ch}' -> вершина {nxt}:")

            f = self.fail[v]
            print(f"    Поиск суффиксной ссылки,
начиная с fail[{v}] = {f}")

            while f > 0 and ch not in self.next[f]:
                print(f"    Переход по fail[{f}] =
{self.fail[f]}")
                f = self.fail[f]

            if ch in self.next[f]:
                self.fail[nxt] = self.next[f][ch]
                print(f"    Найдено: fail[{nxt}] =
{self.fail[nxt]}")
            else:
                self.fail[nxt] = 0
                print(f"    Не найдено: fail[{nxt}] =
0 (корень)")

                if self.out[self.fail[nxt]]:
                    self.term[nxt] = self.fail[nxt]
                    print(f"    Терминальная ссылка:
term[{nxt}] = {self.term[nxt]} (выход в fail)")
                else:
                    self.term[nxt] =
self.term[self.fail[nxt]]
                    print(f"    Терминальная ссылка:
term[{nxt}] = {self.term[nxt]} (наследована)")

            queue.append(nxt)

    print(f"\nПОСТРОЕННЫЙ АВТОМАТ:")
    for i in range(self.vertices):
        print(f"Вершина {i}:")
        print(f"    Переходы: {self.next[i]}")
        print(f"    Суффиксная ссылка: {self.fail[i]}")
        print(f"    Терминальная ссылка: {self.term[i]}")
        print(f"    Выходные подобразцы: {[p + 1 for p in
self.out[i]]}")

    def get_next(self, v, ch):
        if ch not in self.next[v]:
            if v == 0:
                self.next[v][ch] = 0
            else:
                self.next[v][ch] =
self.get_next(self.fail[v], ch)
        return self.next[v][ch]

    def search(self, text, subpatterns):

```

```

print(f"\nПРОЦЕСС ПОИСКА В ТЕКСТЕ: '{text}')
```

results = []
 v = 0
 print(f"Начальное состояние: вершина {v}")

```

for i, ch in enumerate(text):
    print(f"\nПозиция {i + 1}, символ '{ch}':")
    v = self.get_next(v, ch)
    print(f"  Переход в вершину: {v}")
    u = v

    if u > 0:
        print(f"  Проверка терминальных вершин:")

        while u > 0:
            if self.out[u]:
                for pattern_id in self.out[u]:
                    pattern_len =
len(subpatterns[pattern_id])
                    pos = i - pattern_len + 1
                    print(f"          Найден подобразец
{pattern_id + 1} на позиции {pos}")
                    results.append((pos, pattern_id))
                    print(f"          Переход по term[{u}] =
{self.term[u]}")
                    u = self.term[u]

        return results

```

def solve_with_wildcard(text, pattern, wildcard):
 print(f"\nПОИСК С ДЖОКЕРОМ")
 print(f"Текст: '{text}')

print(f"Образец: '{pattern}')

print(f"Символ джокера: '{wildcard}')

```

    if pattern.count(wildcard) == len(pattern):
        print("Ошибка: образец состоит только из джокеров")
        return

    subpatterns = []
    positions = []

    current = ""
    start_pos = None

    print(f"\nРАЗБИЕНИЕ ОБРАЗЦА НА ПОДОБРАЗЦЫ:")
    for i, ch in enumerate(pattern):
        if ch != wildcard:
            if current == "":
                start_pos = i
            current += ch

```

```

        else:
            if current != "":
                subpatterns.append(current)
                positions.append(start_pos)
                print(f"    Подобразец '{current}' на позиции
{start_pos} в образце")
                current = ""

            if current != "":
                subpatterns.append(current)
                positions.append(start_pos)
                print(f"    Подобразец '{current}' на позиции
{start_pos} в образце")

        print(f"\nВсего подобразцов: {len(subpatterns)}")
        print(f"Подобразцы: {subpatterns}")
        print(f"Их позиции в образце: {positions}")

        if len(subpatterns) == 0:
            print("Ошибка: нет подобразцов без джокеров")
            return

        ac = AhoCorasick()
        for i, sp in enumerate(subpatterns):
            ac.add_pattern(sp, i)
        ac.build_automaton()

        occurrences = ac.search(text, subpatterns)

        pattern_len = len(pattern)
        text_len = len(text)
        counter = [0] * (text_len + 1)

        print(f"\nПРОВЕРКА СОВПАДЕНИЙ ОБРАЗЦА С ДЖОКЕРАМИ")
        print(f"Длина образца: {pattern_len}")
        print(f"Необходимо найти все {len(subpatterns)}
подобразцов на правильных позициях")

        results = []

        for pos, pat_id in occurrences:
            start = pos - positions[pat_id]
            print(f"\nПодобразец {pat_id + 1} найден на позиции
{pos}")

            print(f"    Позиция в образце: {positions[pat_id]}")
            print(f"    Предполагаемое начало образца: {start}")

            if start >= 0 and start + pattern_len <= text_len:
                counter[start] += 1
                print(f"    Счетчик совпадений для позиции {start
+ 1}: {counter[start]}/{len(subpatterns)}")

            if counter[start] == len(subpatterns):

```

```

        print(f"      НАЙДЕНО ПОЛНОЕ СОВПАДЕНИЕ на
позиции {start + 1}")
        results.append(start + 1)
    else:
        print(f"      Выход за границы текста")

    print(f"\nИТОГОВЫЕ ПОЗИЦИИ ВХОЖДЕНИЙ:")
    if results:
        for pos in results:
            print(pos)
    else:
        print("Вхождения не найдены")

if __name__ == "__main__":
    text = input().strip()
    pattern = input().strip()
    wildcard = input().strip()

    solve_with_wildcard(text, pattern, wildcard)

```